CSC 251 Lecture 13 –Data Visualization 2

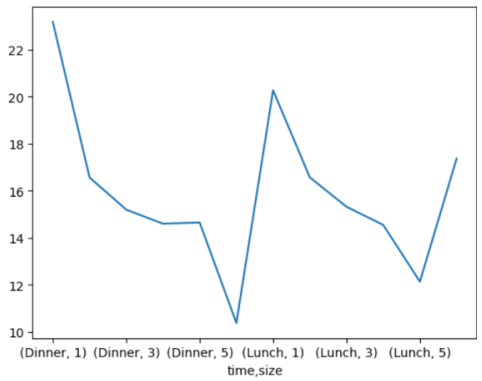
Grouping by Multiple Columns

Example 1a&b: The order does make a difference as the first column will be used for the x-axis and the second for the legend. Note for our tips dataset:

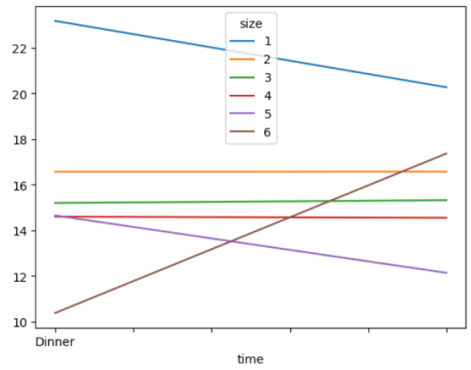
```
timesize = tips.groupby(['time', 'size'])['percent'].mean()
```

```
time    size
Dinner  1       23.183197
        2       16.570395
        3       15.199487
        4       14.601680
        5       14.652219
        6       10.379905
Lunch   1       20.275206
        2       16.574967
        3       15.322597
        4       14.551515
        5       12.138869
        6       17.370592
Name: percent, dtype: float64
```

`timesize.plot()`



`timesize.unstack().plot()`

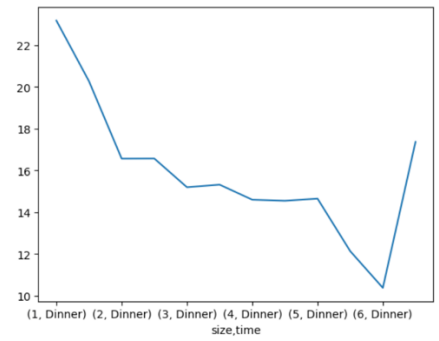


Versus:

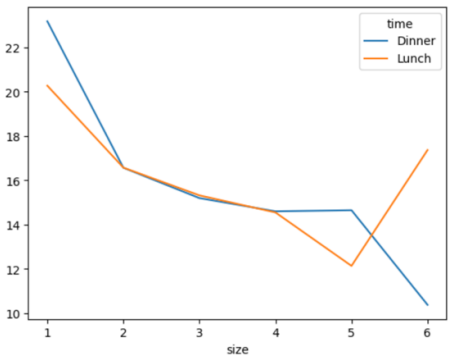
```
sizetime = tips.groupby(['size', 'time'])['percent'].mean()
```

```
size  time
1     Dinner    23.183197
      Lunch     20.275206
2     Dinner    16.570395
      Lunch     16.574967
3     Dinner    15.199487
      Lunch     15.322597
4     Dinner    14.601680
      Lunch     14.551515
5     Dinner    14.652219
      Lunch     12.138869
6     Dinner    10.379905
      Lunch     17.370592
Name: percent, dtype: float64
```

`sizetime.plot()`



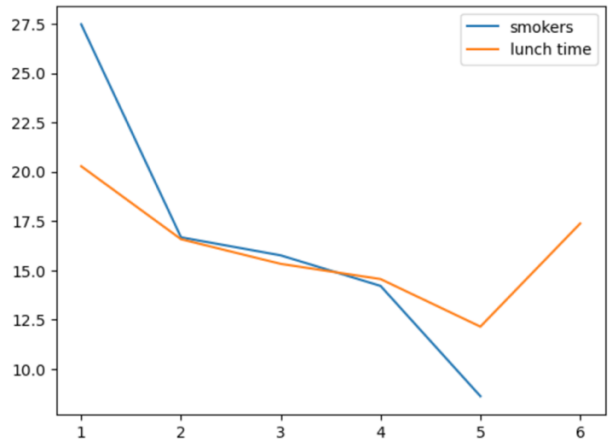
`sizetime.unstack().plot()`



Adding Multiple Plots to the Same Axes

Example 2: We can include multiple plots on the same axes by plotting one after another in the same cell or between `plt.show()` commands in a file. From our tips dataset:

```
smoker= tips[tips['smoker'] == 'Yes' ]
plt.plot(smoker.groupby('size')['percent'].mean(), label = 'smokers')
lunch= tips[tips['time'] == 'Lunch' ]
plt.plot(lunch.groupby('size')['percent'].mean(), label = 'lunch time')
plt.legend()
```

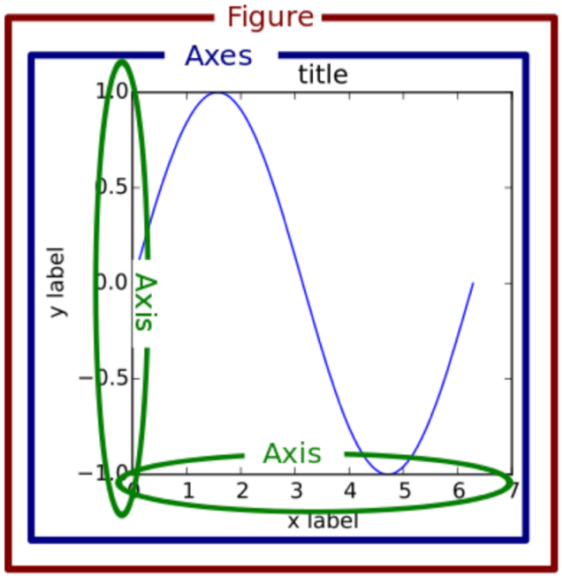


Anatomy of a Figure: The Mystery Behind Simple Plots

What's hidden but implied is that a matplotlib plot is really **a hierarchy of nested Python objects**, so

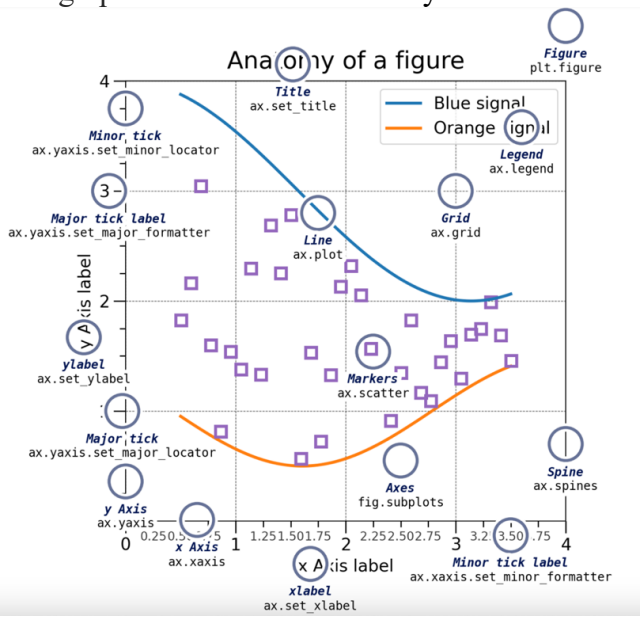
> "plt.plot() is a state-machine interface that implicitly tracks the current figure…"

which implies that you're only creating and manipulating one plot at a time and don't need to explicitly refer to it.
https://realpython.com/python-matplotlib-guide/#why-can-matplotlib-be-confusing



A Figure object is the outmost container for a graphic which can contain multiple Axes objects. An Axes object refers to an individual plot or graph, not to be confused with the plural of "axis" in a plot. Within each Axes object, are smaller objects such as tick marks, lines, legends, text boxes, etc.

The graphic below was created by and is used to describe matplotlib features:



https://matplotlib.org/stable/gallery/showcase/anatomy.html

Note the following:

```
plt.title('…')
plt.xlabel('…')
```

are wrappers for object-oriented methods:

```
plt.set_title('…')
plt.set_xlabel('…')
```

We've created simple plots without explicit reference to the plot:

```
plt.plot(…)
```

We can create a Figure object to contain multiple Axes objects:

```
fig, ax = plt.subplots()
```

where we can include arguments to set the size, the number of plots, etc. Then we can set properties for the Axes object, including multiple plots in the same figure.

A Grid of Axes Objects

Example 3a: The Anscombe Quartet dataset contains 4 sets of data, I – IV, with 11 x- and y-values each.

	dataset	x	y											
0	I	10.0	8.04	11	II	10.0	9.14	22	III	10.0	7.46	33	IV	8.0	6.58
1	I	8.0	6.95	12	II	8.0	8.14	23	III	8.0	6.77	34	IV	8.0	5.76
2	I	13.0	7.58	13	II	13.0	8.74	24	III	13.0	12.74	35	IV	8.0	7.71
3	I	9.0	8.81	14	II	9.0	8.77	25	III	9.0	7.11	36	IV	8.0	8.84
4	I	11.0	8.33	15	II	11.0	9.26	26	III	11.0	7.81	37	IV	8.0	8.47
5	I	14.0	9.96	16	II	14.0	8.10	27	III	14.0	8.84	38	IV	8.0	7.04
6	I	6.0	7.24	17	II	6.0	6.13	28	III	6.0	6.08	39	IV	8.0	5.25
7	I	4.0	4.26	18	II	4.0	3.10	29	III	4.0	5.39	40	IV	19.0	12.50
8	I	12.0	10.84	19	II	12.0	9.13	30	III	12.0	8.15	41	IV	8.0	5.56
9	I	7.0	4.82	20	II	7.0	7.26	31	III	7.0	6.42	42	IV	8.0	7.91
10	I	5.0	5.68	21	II	5.0	4.74	32	III	5.0	5.73	43	IV	8.0	6.89

Statistically, the data sets appear identical:

```
anscombe.groupby('dataset').mean()
```

dataset	x	y
I	9.0	7.500909
II	9.0	7.500909
III	9.0	7.500000
IV	9.0	7.500909

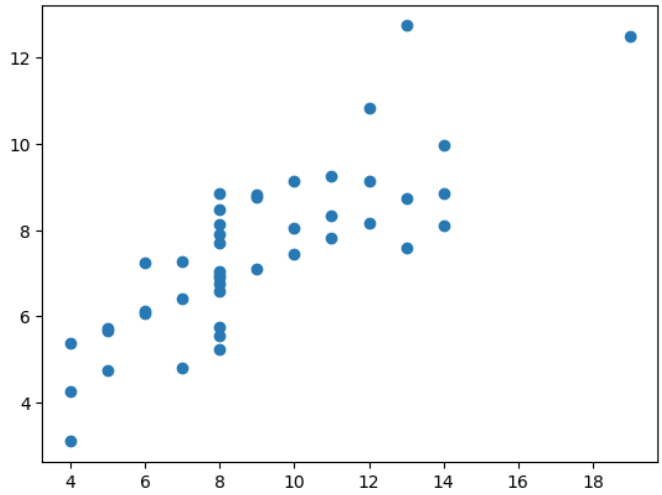
```
anscombe.groupby('dataset').var()
```

dataset	x	y
I	11.0	4.127269
II	11.0	4.127629
III	11.0	4.122620
IV	11.0	4.123249

```
anscombe.groupby('dataset').corr()
```

dataset		x	y
I	x	1.000000	0.816421
	y	0.816421	1.000000
II	x	1.000000	0.816237
	y	0.816237	1.000000
III	x	1.000000	0.816287
	y	0.816287	1.000000
IV	x	1.000000	0.816521
	y	0.816521	1.000000

```
plt.plot(anscombe['x'], anscombe['y'],'o')
```



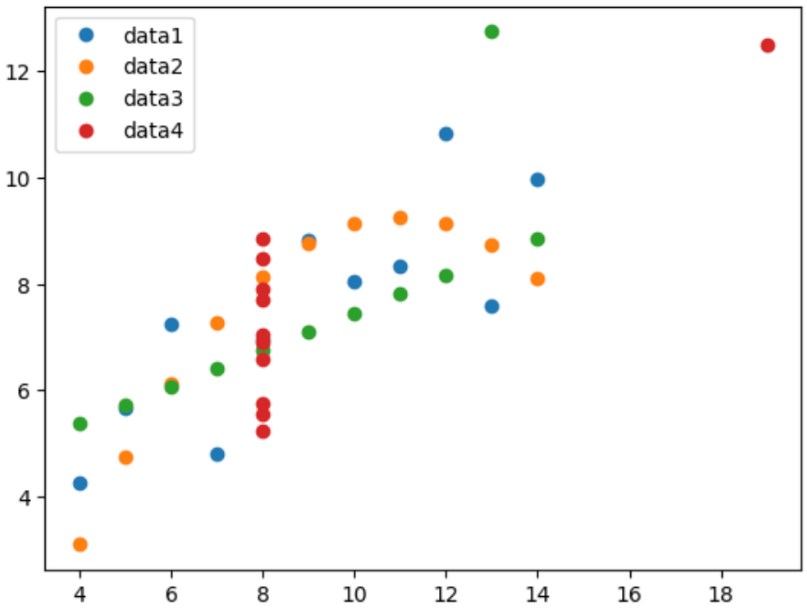
Are the datasets truly identical?

Example 3b: Create subsets of the data by dataset:

```
data1 = anscombe[anscombe['dataset'] == 'I']
data2 = anscombe[anscombe['dataset'] == 'II']
data3 = anscombe[anscombe['dataset'] == 'III']
data4 = anscombe[anscombe['dataset'] == 'IV']
```

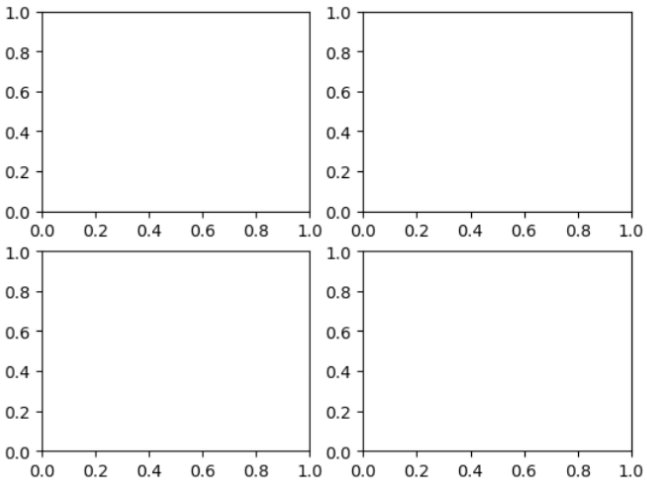
Plot on the same Axes:

```
fig, axes = plt.subplots()
axes.plot(data1['x'], data1['y'], 'o', label='data1')
axes.plot(data2['x'], data2['y'], 'o', label='data2')
axes.plot(data3['x'], data3['y'], 'o', label='data3')
axes.plot(data4['x'], data4['y'], 'o', label='data4')
axes.legend(loc='best')
```



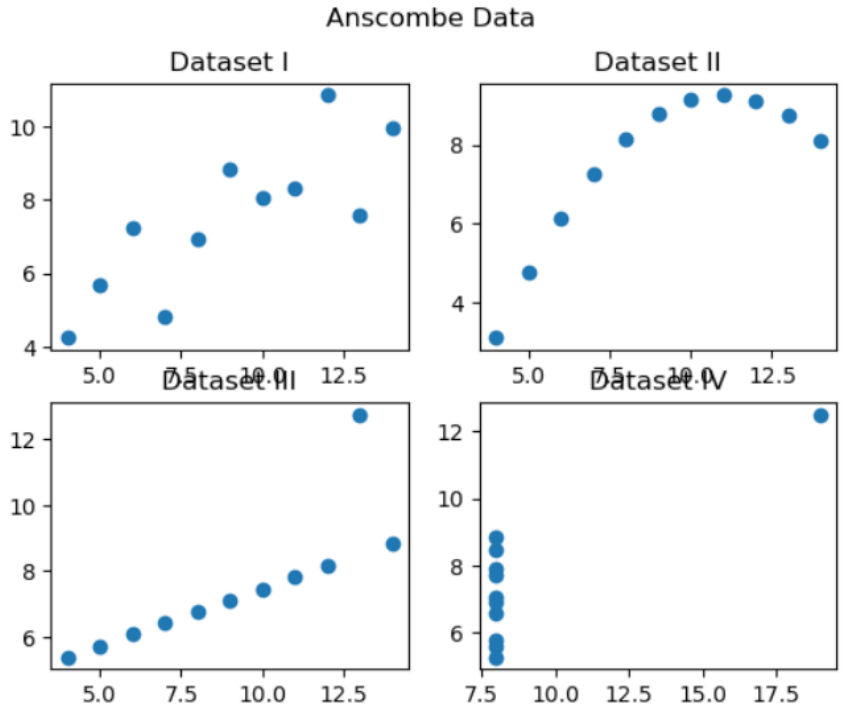
Example 3c: Instead, let's create a figure and split it into a 2x2 grid of plots:

```
fig = plt.figure()
axes1 = fig.add_subplot(2, 2, 1)
axes2 = fig.add_subplot(2, 2, 2)
axes3 = fig.add_subplot(2, 2, 3)
axes4 = fig.add_subplot(2, 2, 4)
```

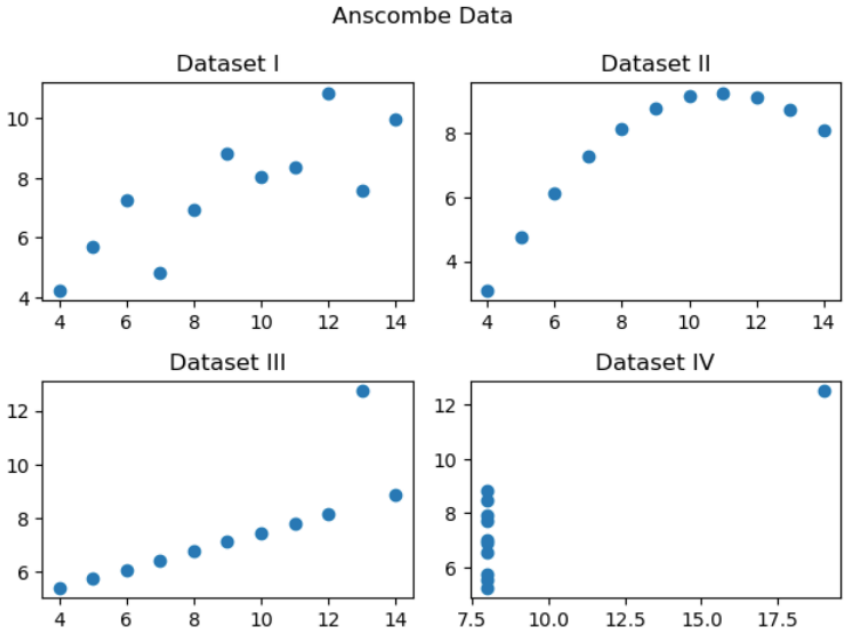


Create a plot on each Axes and add a few labels, and "ta dah!" (or not!):

```
axes1.plot(data1['x'], data1['y'], 'o')
axes2.plot(data2['x'], data2['y'], 'o')
axes3.plot(data3['x'], data3['y'], 'o')
axes4.plot(data4['x'], data4['y'], 'o')
fig.suptitle("Anscombe Data")
axes1.set_title("Dataset I")
axes2.set_title("Dataset II")
axes3.set_title("Dataset III")
axes4.set_title("Dataset IV")
```



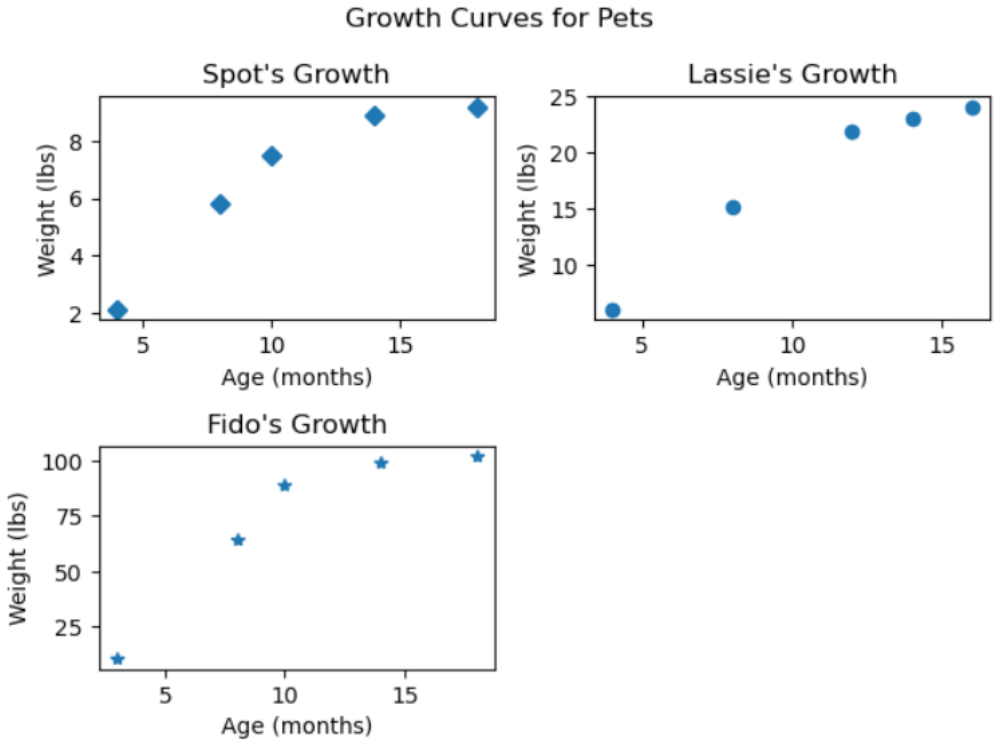
There's a method to adjust the spacing between the Axes:
`fig.tight_layout()`



Are the datasets identical?

Example 4: Generally, the data on multiple graphs within the same figure should contain the same variable on the axes (maybe not the same scale) or should consist of plots of different aspects of the same dataset.

Another example from our pets dataset:

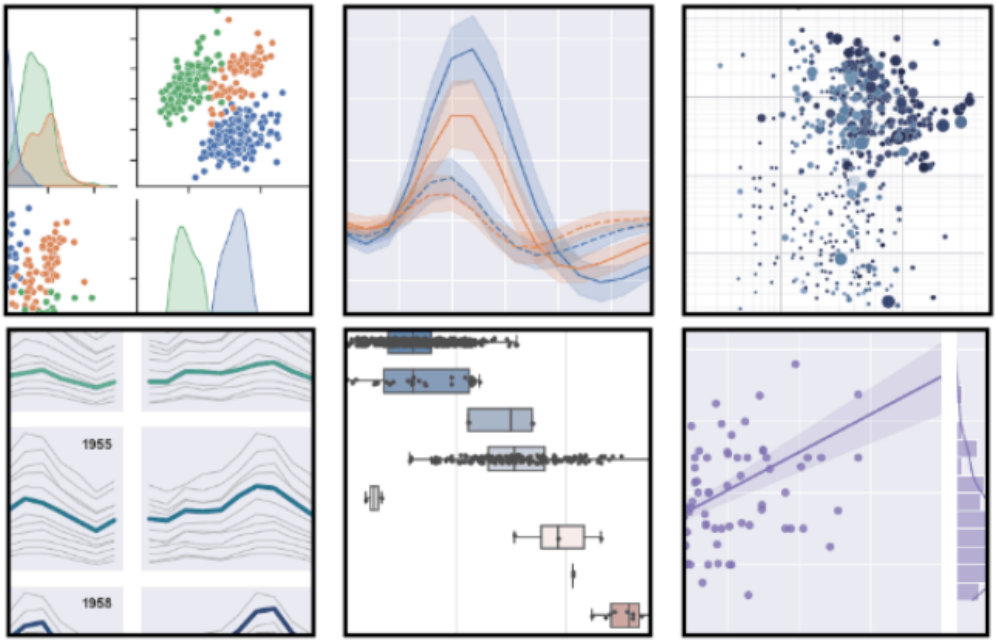


Seaborn

Seaborn builds upon matplotlib and is integrated with pandas to create a higher-level interface for statistical graphs. This makes some visualizations easier to plot! https://seaborn.pydata.org/

```
import seaborn as sns
```

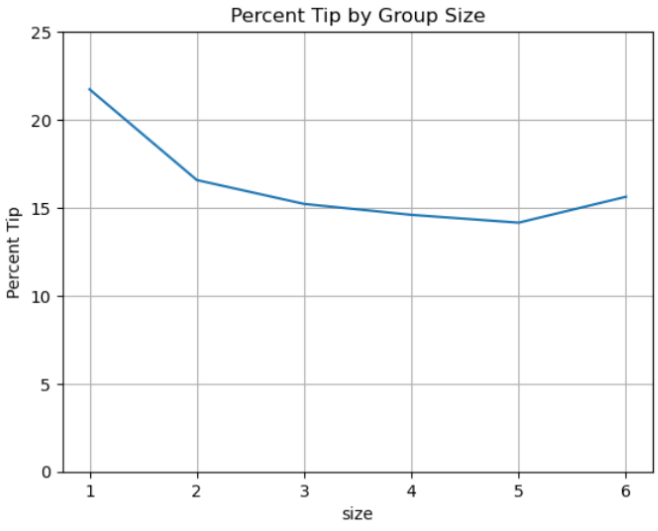
Seaborn examples:



Plot Types

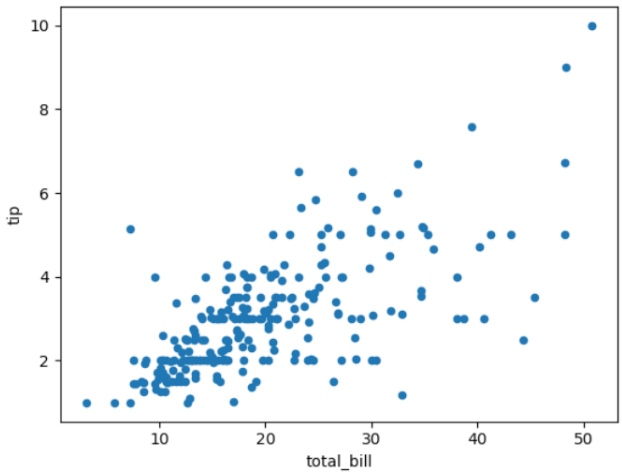
1) **Line plot** – shows the relationship between two or more columns of data, often showing changes over time. Pandas allows you to customize within a single line of script:

```
groupsize.plot.line(title='Percent Tip by Group Size', ylabel='Percent Tip',
      ylim=(0,25), grid=True)
```



2) **Scatter plot** – shows relationships between two columns of data, must specify x and y parameters.

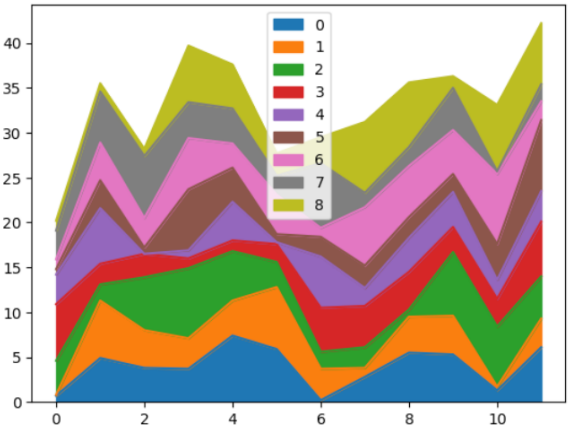
```
tips.plot.scatter(x = 'total_bill', y = 'tip')
```

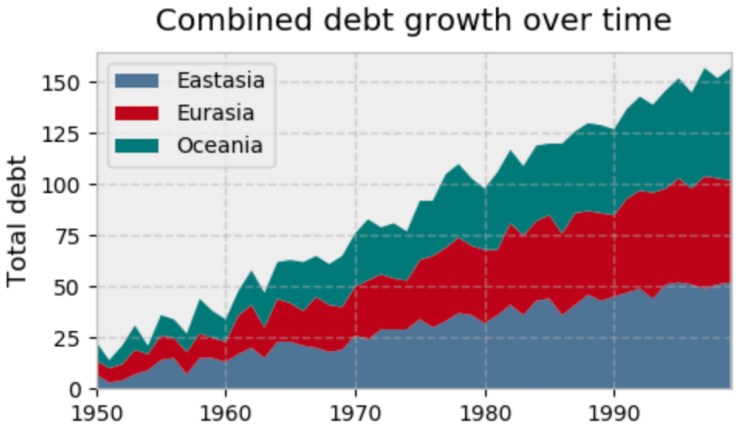


Useful to investigate correlations between variables for EDA!

3) **Area plot** – a line plot with data in each column added to the data in the previous column.

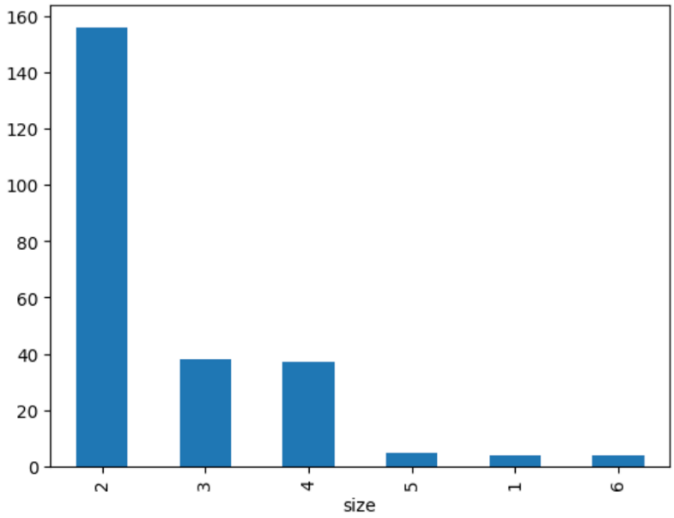
From the Woolly Worm data: `wwdf.plot.area()`





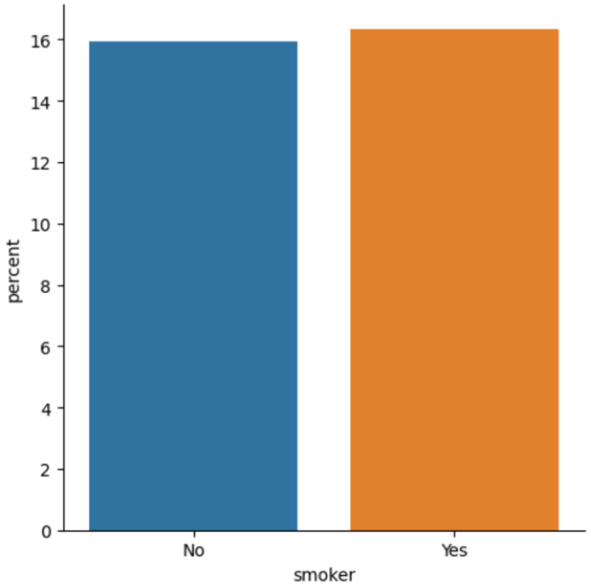
4) **Bar graph** – shows data values for different categories

```
tipscount = tips['size'].value_counts().plot(kind = 'bar')
```



Or using Seaborn:

```
sns.catplot(data=tips, kind = 'bar', x = 'smoker', y ='percent', errorbar = None)
```



What observation might we make regarding the tips data?

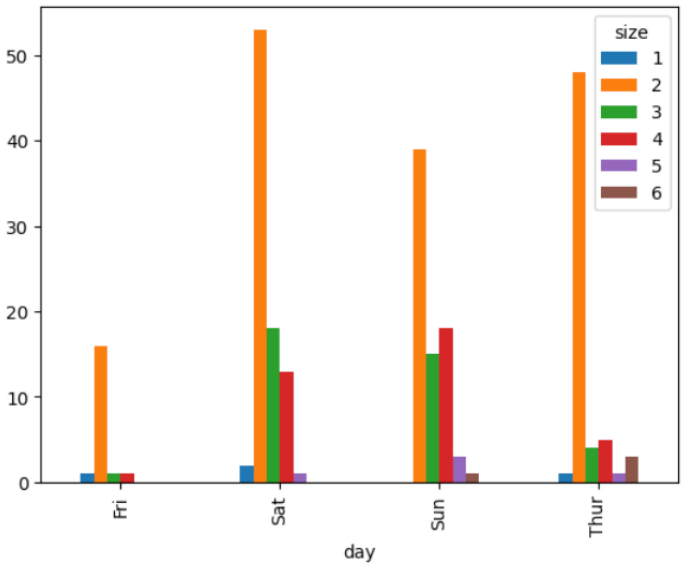
Create a frequency table:

```
group_count = pd.crosstab(tips['day'], tips['size'])
```

size	1	2	3	4	5	6
day						
Fri	1	16	1	1	0	0
Sat	2	53	18	13	1	0
Sun	0	39	15	18	3	1
Thur	1	48	4	5	1	3

What observation might we make regarding the tips data?

```
group_count.plot.bar()
```



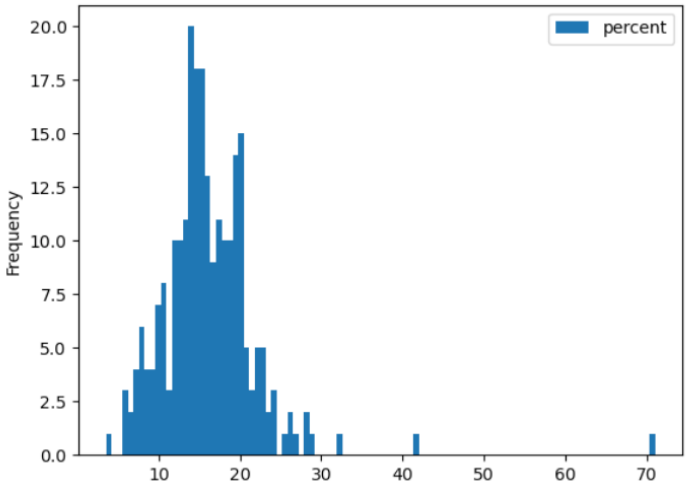
5) **Histogram** – shows the frequency of that datapoints in a numeric column by dividing the data into groups, or bins, along the x-axis.

What can we learn with this histogram?

```
plt.hist(tips['percent'], bins=100)
```

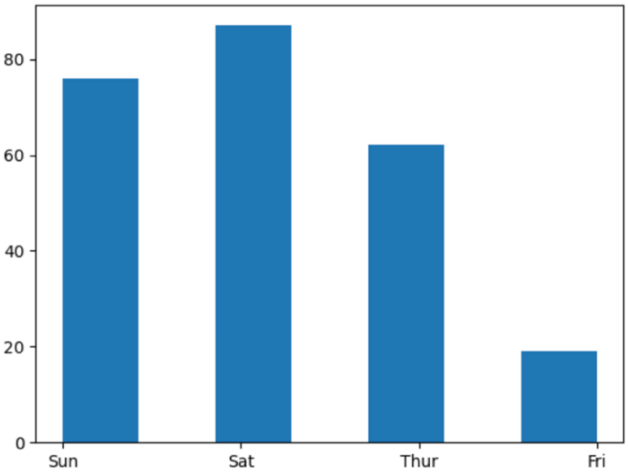
or

```
tips.plot.hist(y='percent', bins=100)
```



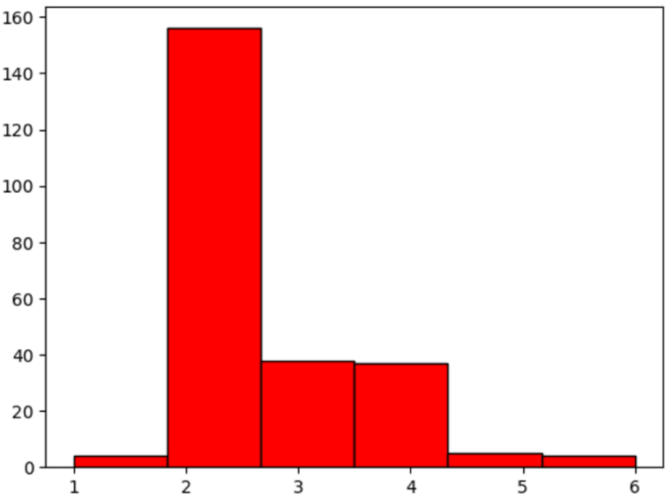
What does this tell us?

```
plt.hist(tips['day'], bins = 7)
```



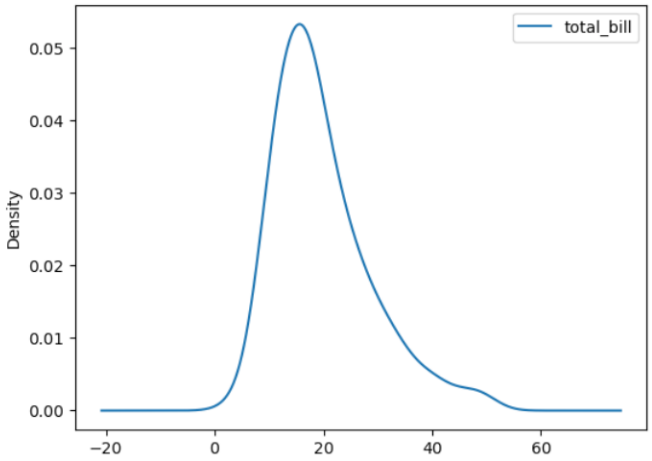
You can customize your histograms. Notice the difference between this one and the previous one?

```
plt.hist(tips['size'], bins = 6, facecolor = 'red', edgecolor = 'black')
```



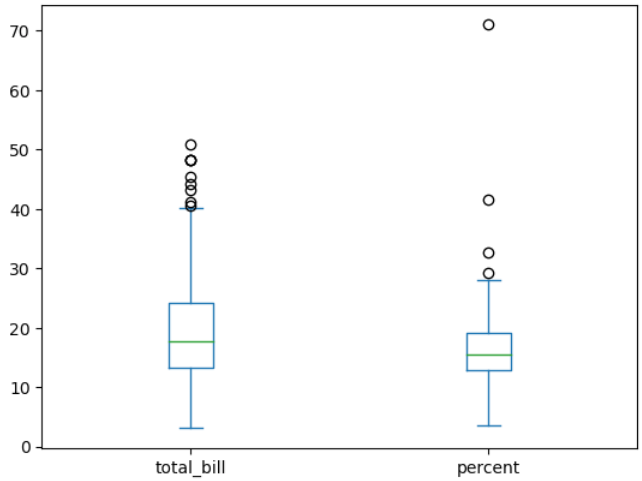
6) **Density plot** – shows the distribution of data in a numeric column using a kernel density estimate to smooth the data in a histogram.

```
tips.plot.density(y ='total_bill')
```



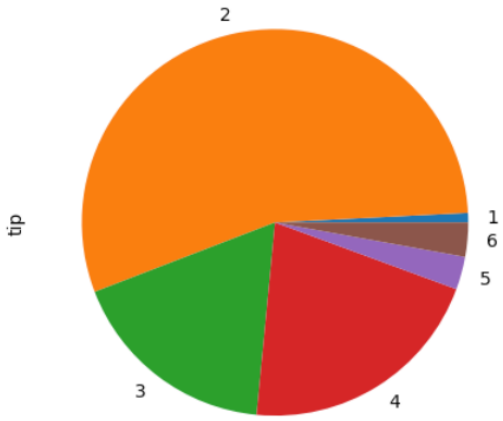
7) **Box plot** – illustrates the distribution with the box spanning the 1st through 3rd quartiles, line marking the median, outliers marked by circles. What does this confirm?

```
subtips = tips[['total_bill', 'percent']]
subtips.plot.box()
```

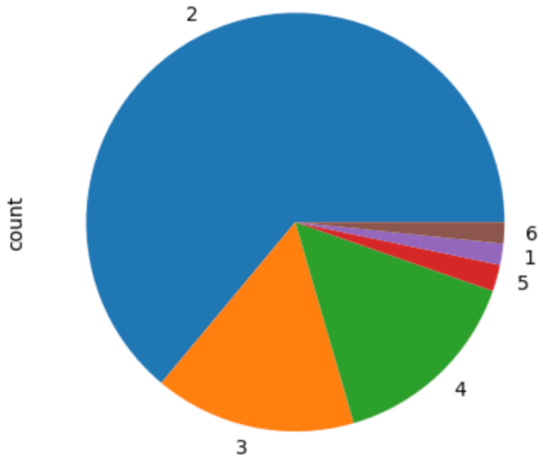


8) **Pie plot** – shows proportions of categories that make up the whole.

```
tips.groupby('size')['tip'].sum().plot.pie()
```



```
tips['size'].value_counts().plot.pie()
```



Interactive Plotting: Plotly

Plotly is an open-source, interactive, browser-based graphing library. Best for interactive notebooks and presentations, but maybe not for static reports. Some functions require a response from Plotly's server.

Example 5: Once installed, the plotly express module is a good place to start, allowing you to create over 30 types of graphs. Can serve as a backend for pandas plotting with dataframes or can be imported:

```
import plotly.express as px

tdf = tips.groupby('size')['tip'].mean
fig = px.bar(tips, x="size", y="tip")
fig.show()
```

Then plot, taking advantage of hover effects in notebook:

